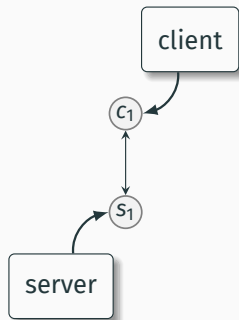


Verifying a high-performance distributed transaction system using permissioned state machines

Yun-Sheng Chang Joseph Tassarotti[†] Frans Kaashoek Nickolai Zeldovich

MIT CSAIL [†]NYU

Distributed transaction systems are complex



Distributed transaction systems are complex

Crash recovery

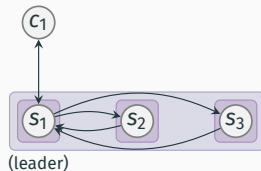


Distributed transaction systems are complex



Crash recovery

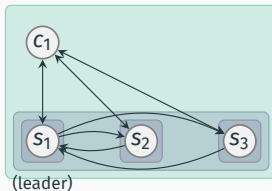
Distributed transaction systems are complex



Crash recovery

Paxos replication

Distributed transaction systems are complex

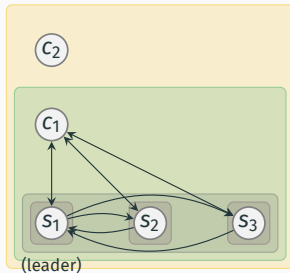


Crash recovery

Paxos replication

Inconsistent replication (IR)

Distributed transaction systems are complex



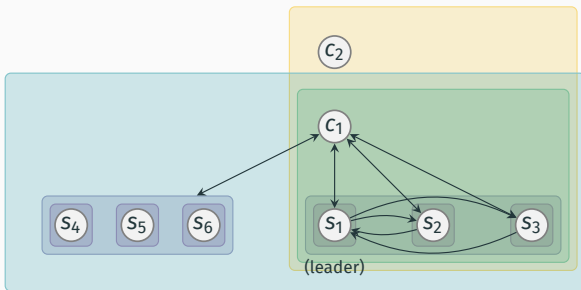
Crash recovery

Paxos replication

Inconsistent replication (IR)

Multi-version concurrency control (MVCC)

Distributed transaction systems are complex



Crash recovery

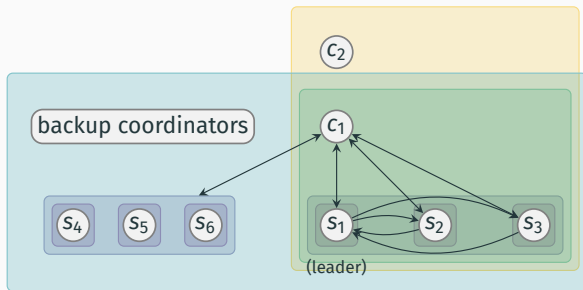
Paxos replication

Inconsistent replication (IR)

Multi-version concurrency control (MVCC)

Fault-tolerant two-phase commit

Distributed transaction systems are complex



Crash recovery

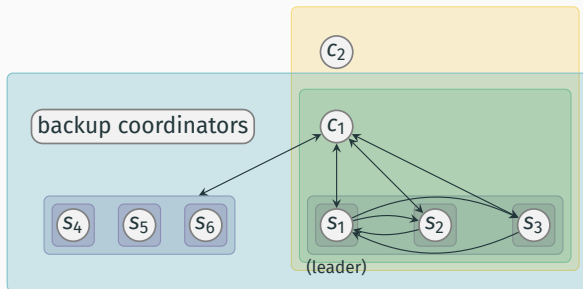
Paxos replication

Inconsistent replication (IR)

Multi-version concurrency control (MVCC)

Fault-tolerant two-phase commit

Distributed transaction systems are complex



Crash recovery

Paxos replication

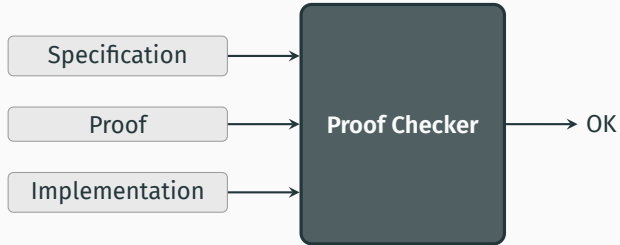
Inconsistent replication (IR)

Multi-version concurrency control (MVCC)

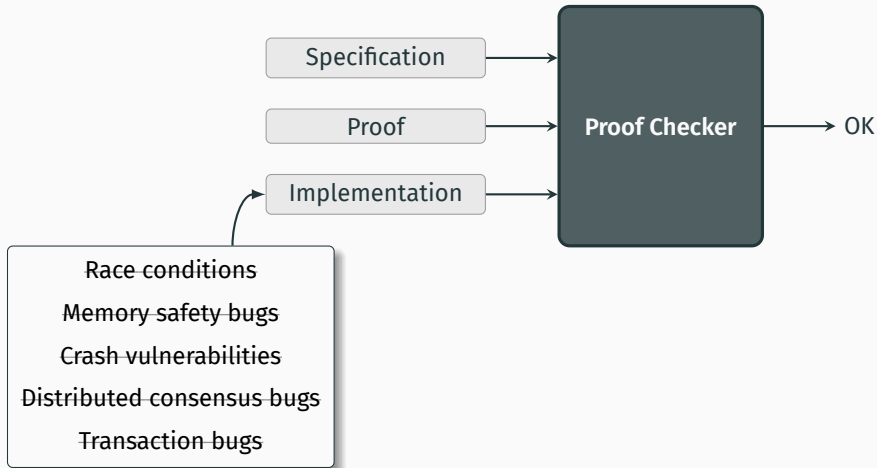
Fault-tolerant two-phase commit

Distributed transaction systems are complex and therefore bug-prone

Approach: formal verification



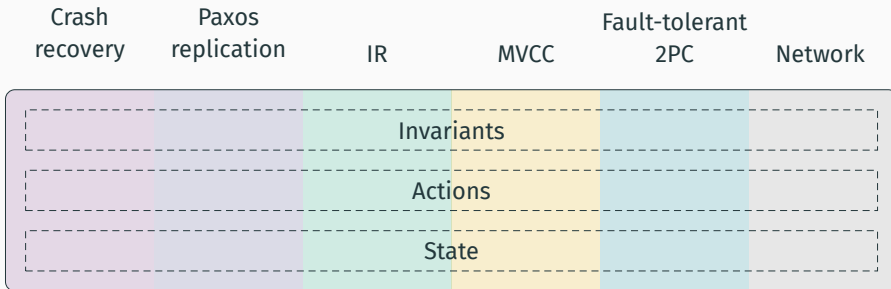
Approach: formal verification



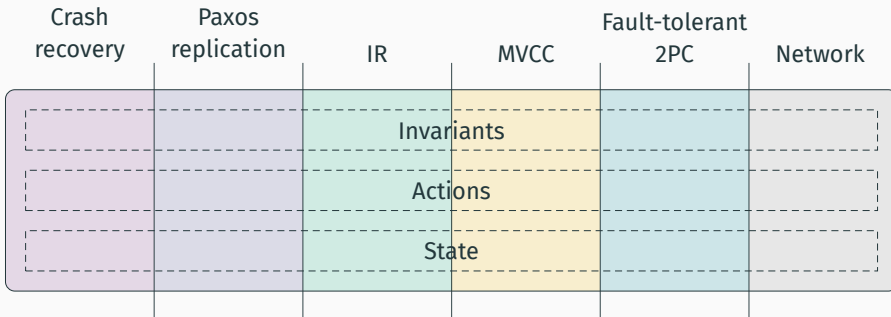
Verifying distributed systems with TLA-style state-machine reasoning



Verifying distributed systems with TLA-style state-machine reasoning

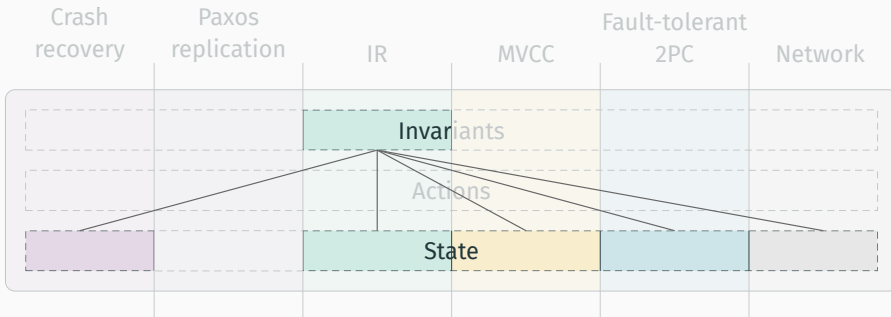


Verifying distributed systems with TLA-style state-machine reasoning



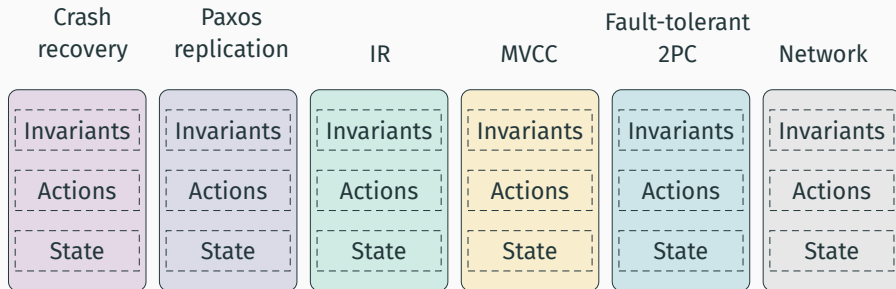
Ideally, we want to reason about each component incrementally and independently

Verifying distributed systems with TLA-style state-machine reasoning



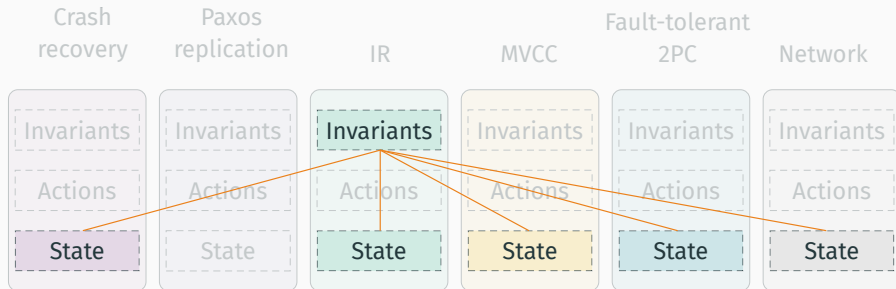
However, invariants or actions over the **shared state** pose modularity challenges

Verifying distributed systems with TLA-style state-machine reasoning



How can we decompose the components into separate modules

Verifying distributed systems with TLA-style state-machine reasoning



How can we decompose the components into separate modules while capturing **inter-component dependencies**?

Main contributions of this work

Permissioned state machines (PSM) = State-machine reasoning
+
Concurrent separation logic (CSL)

Main contributions of this work

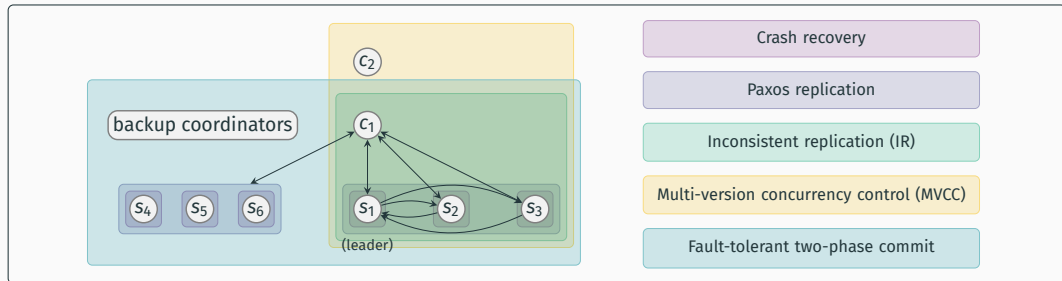
Permissioned state machines (PSM) = State-machine reasoning
+
Concurrent separation logic (CSL)

Case study of applying **PSM** to verifying the **Tulip** distributed transaction system

Main contributions of this work

$$\text{Permissioned state machines (PSM)} = \text{State-machine reasoning} + \text{Concurrent separation logic (CSL)}$$

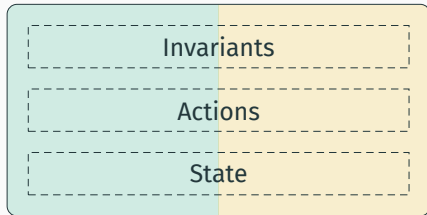
Case study of applying **PSM** to verifying the **Tulip** distributed transaction system



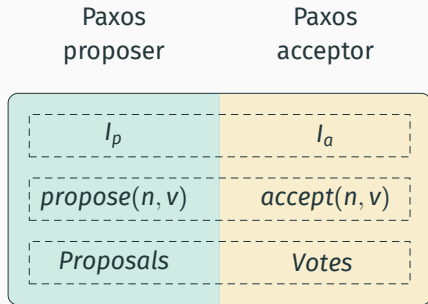
Example: decomposing single-decree Paxos into modules

Paxos
proposer

Paxos
acceptor



Example: decomposing single-decree Paxos into modules



Example: decomposing single-decree Paxos into modules

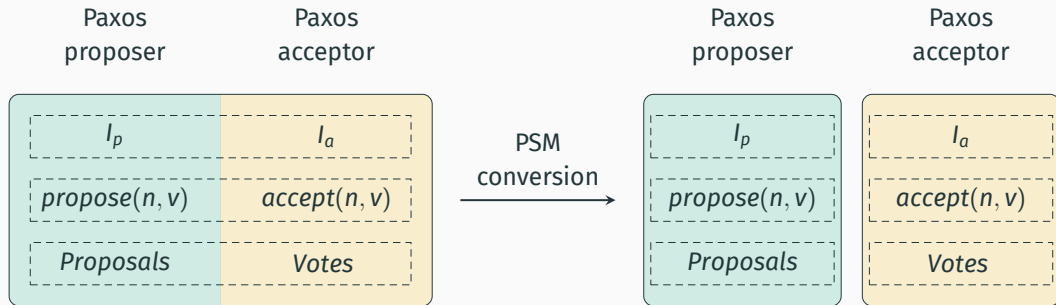


Illustration of the single-decree Paxos protocol (incomplete)

Proposers

p_1

Acceptors

a_1

a_2

a_3

Illustration of the single-decree Paxos protocol (incomplete)

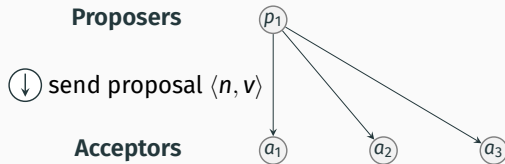


Illustration of the single-decree Paxos protocol (incomplete)

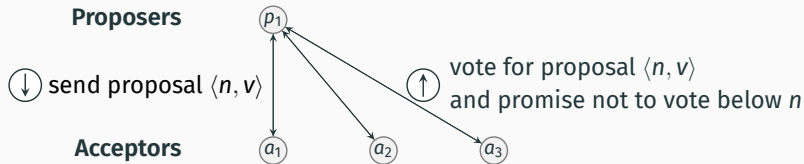
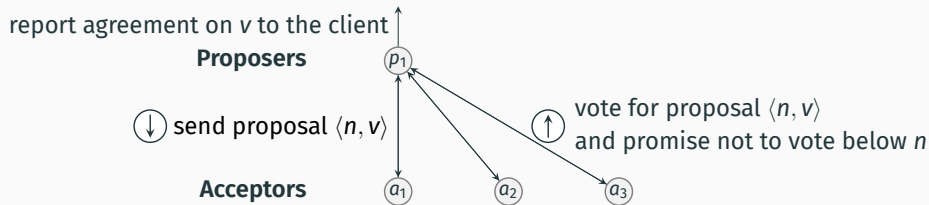


Illustration of the single-decree Paxos protocol (incomplete)



Formulating Paxos as a state machine—state, action, and invariant

Proposals (proposer state)

num	val
<i>m</i>	<i>u</i>

Formulating Paxos as a state machine—state, action, and invariant

Proposals (proposer state)

num	val
m	u

Votes (acceptor state)



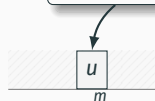
Formulating Paxos as a state machine—state, action, and invariant

Proposals (proposer state)

num	val
m	u

Votes (

$\langle m, u \rangle$ accepted by the acceptor



Formulating Paxos as a state machine—state, action, and invariant

Proposals (proposer state)

num	val
m	u

Votes (acceptor state)



otherwise rejected by the acceptor

Formulating Paxos as a state machine—state, action, and invariant

Proposals (proposer state)

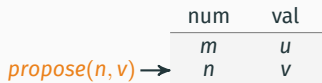
num	val
m	u

Votes (acceptor state)



Formulating Paxos as a state machine—state, action, and invariant

Proposals (proposer state)

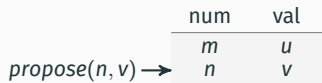


Votes (acceptor state)

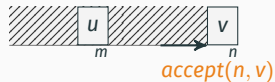


Formulating Paxos as a state machine—state, action, and invariant

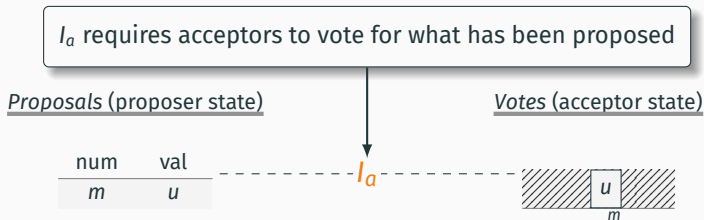
Proposals (proposer state)



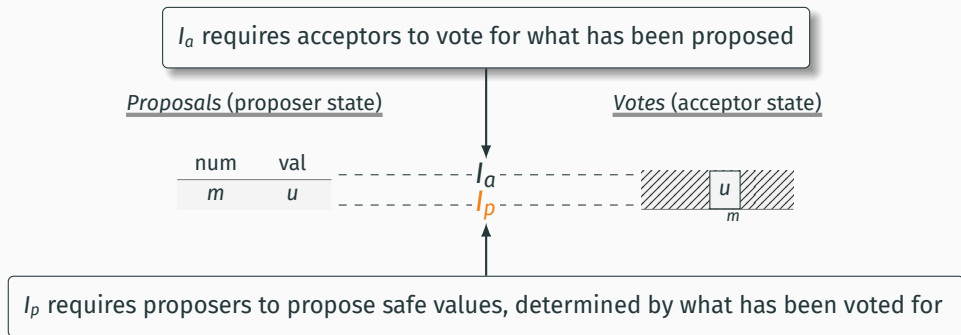
Votes (acceptor state)



Formulating Paxos as a state machine—state, action, and invariant



Formulating Paxos as a state machine—state, action, and invariant

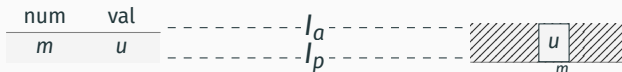


Reasoning about correctness of Paxos—invariance

Correctness is formulated as invariance proofs that **each action preserves each invariant**

Proposals (proposer state)

Votes (acceptor state)

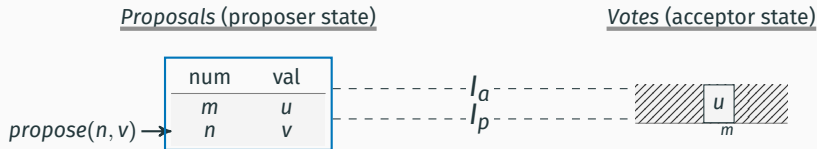


	<i>propose</i> (<i>n</i> , <i>v</i>)	<i>accept</i> (<i>n</i> , <i>v</i>)
I_a		
I_p		

Invariance proofs

Reasoning about correctness of Paxos—invariance

Correctness is formulated as invariance proofs that each action preserves each invariant



	$propose(n, v)$	$accept(n, v)$
I_a	$I_a(\text{Proposals}', \text{Votes})$	
I_p	$I_p(\text{Proposals}', \text{Votes})$	

Invariance proofs

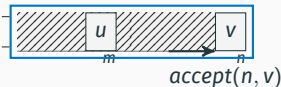
Reasoning about correctness of Paxos—invariance

Correctness is formulated as invariance proofs that each action preserves each invariant

Proposals (proposer state)

num	val
m	u
n	v

Votes (acceptor state)



	$propose(n, v)$	$accept(n, v)$
I_a	$I_a(\text{Proposals}', \text{Votes})$	$I_a(\text{Proposals}, \text{Votes}')$
I_p	$I_p(\text{Proposals}', \text{Votes})$	$I_p(\text{Proposals}, \text{Votes}')$

Invariance proofs

Reasoning about correctness of Paxos—invariance

Tulip: 25 actions

	<i>read</i>	<i>learn</i>	<i>linearize</i>	<i>submit</i>	<i>validate</i>	<i>prepare</i>	<i>commit</i>	<i>abort</i>	<i>execute</i>	<i>...</i>

Reasoning about correctness of Paxos—invariance

Tulip: 25 actions $\times$ $>$ 50 invariants

	<i>read</i>	<i>learn</i>	<i>linearize</i>	<i>submit</i>	<i>validate</i>	<i>prepare</i>	<i>commit</i>	<i>abort</i>	<i>execute</i>	<i>...</i>
<i>l</i> ₁										
<i>l</i> ₂										
<i>l</i> ₃										
<i>l</i> ₄										
<i>l</i> ₅										
<i>l</i> ₆										
<i>l</i> ₇										
<i>l</i> ₈										
$\vdots$										

Reasoning about correctness of Paxos—invariance

Tulip: 25 actions $\times$ $>$ 50 invariants

	<i>read</i>	<i>learn</i>	<i>linearize</i>	<i>submit</i>	<i>validate</i>	<i>prepare</i>	<i>commit</i>	<i>abort</i>	<i>execute</i>	$\dots$
l_1	$l_1(S')$	$l_1(S')$	$l_1(S')$	$l_1(S')$	$l_1(S')$	$l_1(S')$	$l_1(S')$	$l_1(S')$	$l_1(S')$	$\dots$
l_2	$l_2(S')$	$l_2(S')$	$l_2(S')$	$l_2(S')$	$l_2(S')$	$l_2(S')$	$l_2(S')$	$l_2(S')$	$l_2(S')$	$\dots$
l_3	$l_3(S')$	$l_3(S')$	$l_3(S')$	$l_3(S')$	$l_3(S')$	$l_3(S')$	$l_3(S')$	$l_3(S')$	$l_3(S')$	$\dots$
l_4	$l_4(S')$	$l_4(S')$	$l_4(S')$	$l_4(S')$	$l_4(S')$	$l_4(S')$	$l_4(S')$	$l_4(S')$	$l_4(S')$	$\dots$
l_5	$l_5(S')$	$l_5(S')$	$l_5(S')$	$l_5(S')$	$l_5(S')$	$l_5(S')$	$l_5(S')$	$l_5(S')$	$l_5(S')$	$\dots$
l_6	$l_6(S')$	$l_6(S')$	$l_6(S')$	$l_6(S')$	$l_6(S')$	$l_6(S')$	$l_6(S')$	$l_6(S')$	$l_6(S')$	$\dots$
l_7	$l_7(S')$	$l_7(S')$	$l_7(S')$	$l_7(S')$	$l_7(S')$	$l_7(S')$	$l_7(S')$	$l_7(S')$	$l_7(S')$	$\dots$
l_8	$l_8(S')$	$l_8(S')$	$l_8(S')$	$l_8(S')$	$l_8(S')$	$l_8(S')$	$l_8(S')$	$l_8(S')$	$l_8(S')$	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Avoiding action-invariant product with stable permissions

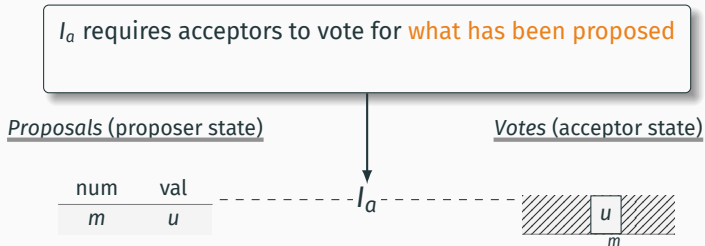
Avoiding action-invariant product with stable permissions

1. Identify **stable invariants**—invariants that remain true under extension

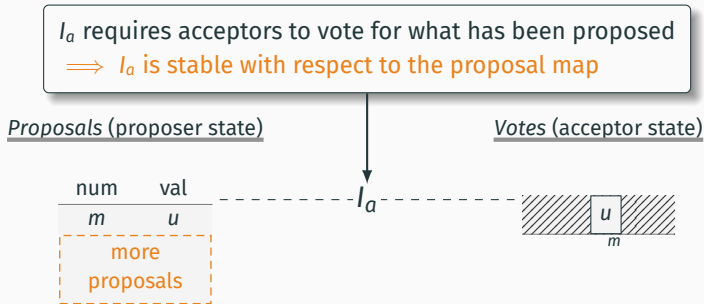
Avoiding action-invariant product with stable permissions

1. Identify stable invariants—invariants that remain true under extension
2. Define stable invariants over **stable permissions** capturing history state as subsets, prefixes, or some other lower bounds

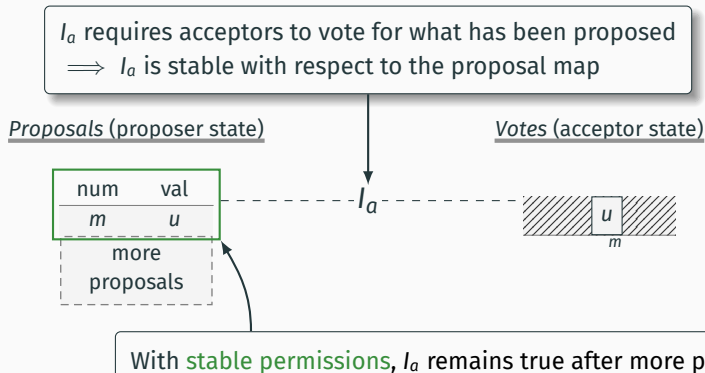
Capturing stable invariants with stable permissions



Capturing stable invariants with stable permissions



Capturing stable invariants with stable permissions



Capturing stable invariants with stable permissions



	$propose(n, v)$	$accept(n, v)$
I_a	$I_a(\text{Proposals}', \text{Votes})$	
I_p		

Invariance proofs

Capturing stable invariants with stable permissions

I_p requires proposers to propose safe values, determined by what has been voted for

Proposals (proposer state)

Votes (acceptor state)



	$propose(n, v)$	$accept(n, v)$
I_a	$I_a(\text{Proposals}', \text{Votes})$	
I_p		

Invariance proofs

Capturing stable invariants with stable permissions

I_p requires proposers to propose safe values, determined by what has been voted for
 $\Rightarrow I_p$ is stable with respect to the vote list

Proposals (proposer state)

Votes (acceptor state)



	$propose(n, v)$	$accept(n, v)$
I_a	$I_a(\text{Proposals}', \text{Votes})$	
I_p		

Invariance proofs

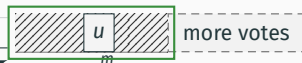
Capturing stable invariants with stable permissions

I_p requires proposers to propose safe values, determined by what has been voted for
 $\Rightarrow I_p$ is stable with respect to the vote list

Proposals (proposer state)

num	val
m	u
n	v

Votes (acceptor state)



With **stable permissions**, I_p remains true after more votes are cast

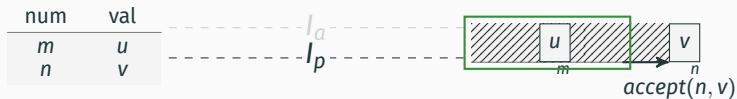
I_a | $I_a(\text{Proposals}', \text{Votes})$
 I_p

Invariance proofs

Capturing stable invariants with stable permissions

Proposals (proposer state)

Votes (acceptor state)



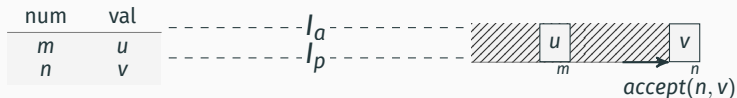
	$propose(n, v)$	$accept(n, v)$
I_a	$I_a(\text{Proposals}', \text{Votes})$	
I_p	$I_p(\text{Proposals}, \text{Votes}')$	

Invariance proofs

Capturing stable invariants with stable permissions

Proposals (proposer state)

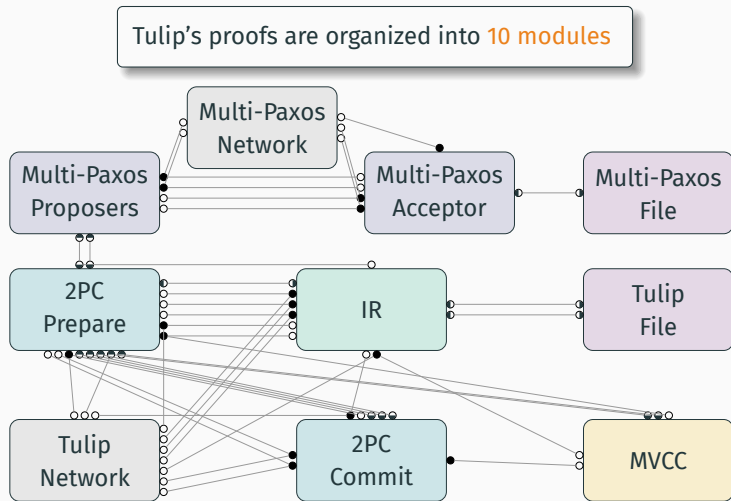
Votes (acceptor state)



	$propose(n, v)$	$accept(n, v)$
I_a	$I_a(\text{Proposals}', \text{Votes})$	$I_a(\text{Proposals}, \text{Votes}')$
I_p	$I_p(\text{Proposals}', \text{Votes})$	$I_p(\text{Proposals}, \text{Votes}')$

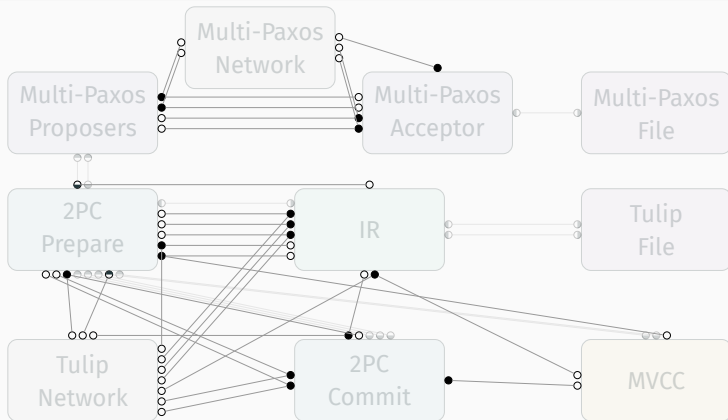
Invariance proofs

A recurring pattern in Tulip's proofs

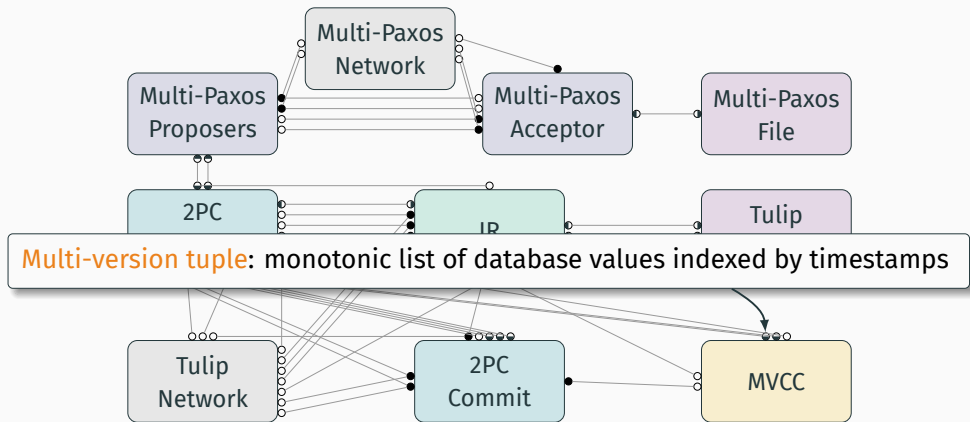


A recurring pattern in Tulip's proofs

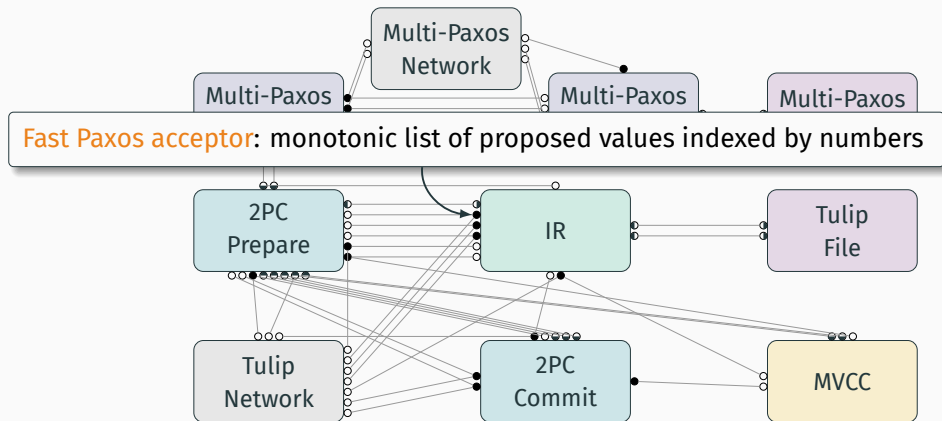
Overall, **13 state variables** in Tulip are shared using stable permissions



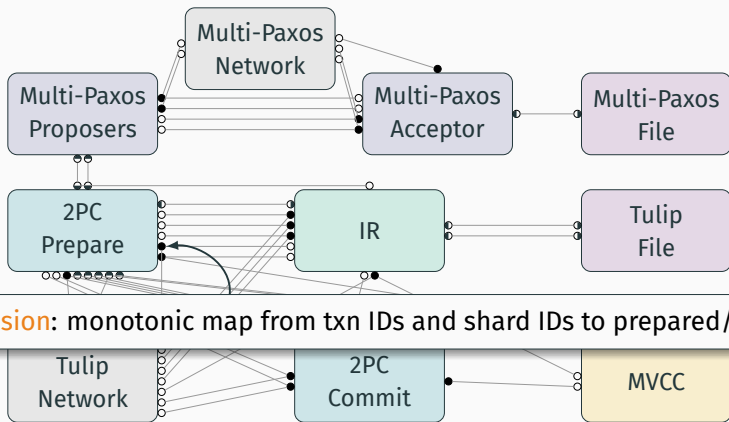
A recurring pattern in Tulip's proofs



A recurring pattern in Tulip's proofs

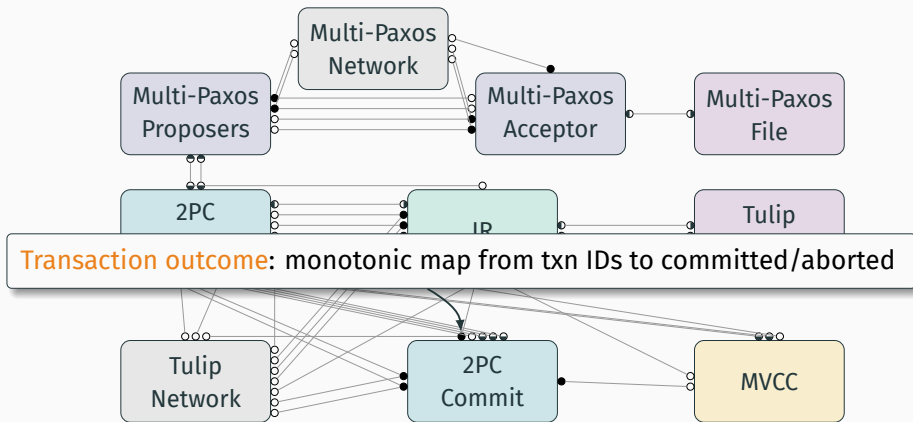


A recurring pattern in Tulip's proofs



Prepare decision: monotonic map from txn IDs and shard IDs to prepared/unprepared

A recurring pattern in Tulip's proofs



Tulip: a formally verified distributed transaction system implementation

Implementation feature and optimization

- TAPIR-style [SOSP '15] inconsistent replication (IR)
- Multi-version concurrency control (MVCC)
- Fault-tolerant two-phase commit (2PC)

Component	Lines of code
Program	3956 (Go)

Tulip: a formally verified distributed transaction system implementation

Implementation feature and optimization

- TAPIR-style [SOSP '15] inconsistent replication (IR)
- Multi-version concurrency control (MVCC)
- Fault-tolerant two-phase commit (2PC)

Component	Lines of code
Program	3956 (Go)
Spec (4 ops)	42 (Rocq)

Tulip: a formally verified distributed transaction system implementation

Implementation feature and optimization

- TAPIR-style [SOSP '15] inconsistent replication (IR)
- Multi-version concurrency control (MVCC)
- Fault-tolerant two-phase commit (2PC)

Proof framework

- Grove [SOSP '23], Iris [JFP '18], and Rocq

Component	Lines of code
Program	3956 (Go)
Spec (4 ops)	42 (Rocq)
Proof	42131 (Rocq)

Tulip: a formally verified distributed transaction system implementation

Implementation feature and optimization

- TAPIR-style [SOSP '15] inconsistent replication (IR)
- Multi-version concurrency control (MVCC)
- Fault-tolerant two-phase commit (2PC)

Proof framework

- Grove [SOSP '23], Iris [JFP '18], and Rocq

Component	Lines of code
Program	3956 (Go)
Spec (4 ops)	42 (Rocq)
Proof	42131 (Rocq)

Proof-to-code ratio: 10.6×

Tulip: a formally verified distributed transaction system implementation

Implementation feature and optimization

- TAPIR-style [SOSP '15] inconsistent replication (IR)
- Multi-version concurrency control (MVCC)
- Fault-tolerant two-phase commit (2PC)

Proof framework

- Grove [SOSP '23], Iris [JFP '18], and Rocq

Component	Lines of code
Program	3956 (Go)
Spec (4 ops)	42 (Rocq)
Proof	42131 (Rocq)

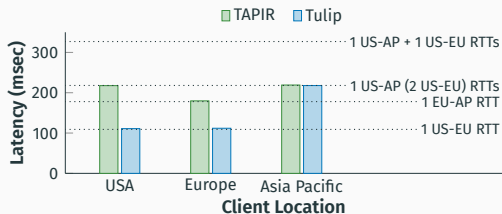
Proof-to-code ratio: 10.6×

Prior work (vKV, GoTxn, CSPEC, CertiKOS): 11–20×

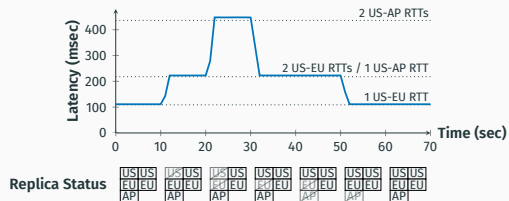
PSM keeps the verification complexity under control

Tulip performance compared with TAPIR

Commit latency



Failure recovery



Compared to TAPIR, Tulip is **verified, faster, able to recover from failures**

PSM marries state-machine reasoning with concurrent separation logic

State-machine reasoning + **Concurrent separation logic**

TLA⁺

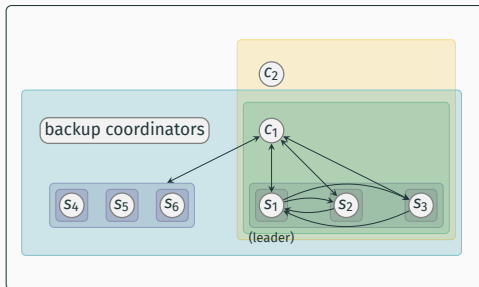
Iris

IronFleet

Grove

Conclusion

Tulip is the first formally verified distributed transaction system implementation, including sophisticated optimizations such as MVCC and IR



Conclusion

Tulip is the first formally verified distributed transaction system implementation, including sophisticated optimizations such as MVCC and IR

Permissioned state machines, and in particular stable permissions, enable modular verification of Tulip

